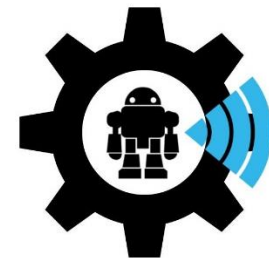




Temas abordados na aula:

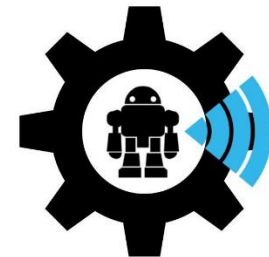
1. Introdução ao ROS;
2. Nível de arquivo de sistemas;
3. Nível de comunidade;
4. Nível grafo computacional;
5. Instalação do ROS;



1. INTRODUÇÃO AO ROS

- Com o avanço da robótica nos dias atuais, realizar a construção de robôs vem se tornando cada vez mais uma tarefa complexa e sofisticada.
- Neste contexto surge o Robot Operating System (ROS) que é um *middleware* de código livre que possui uma série de ferramentas e bibliotecas com o objetivo de facilitar o desenvolvimento de *softwares* para robôs.
- Atualmente existem duas versões do ROS: o ROS e ROS 2. Neste curso não será abordado ROS 2.





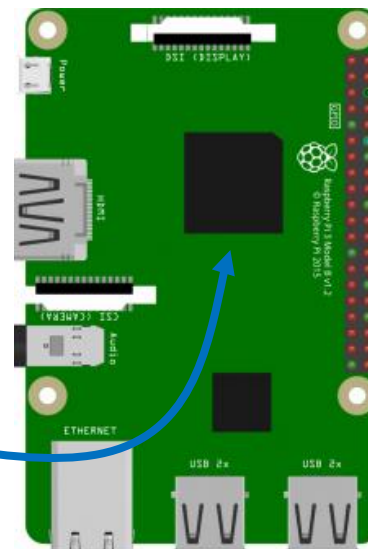
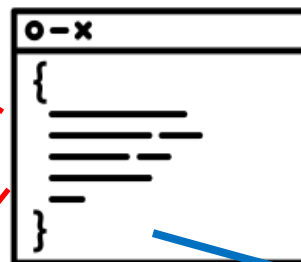
1. INTRODUÇÃO AO ROS

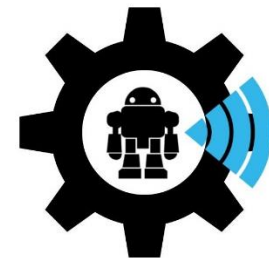
- Principais características:
 - Abstração de *hardware*;

Aquisição de Imagens da PiCamera

Processamento de Imagem

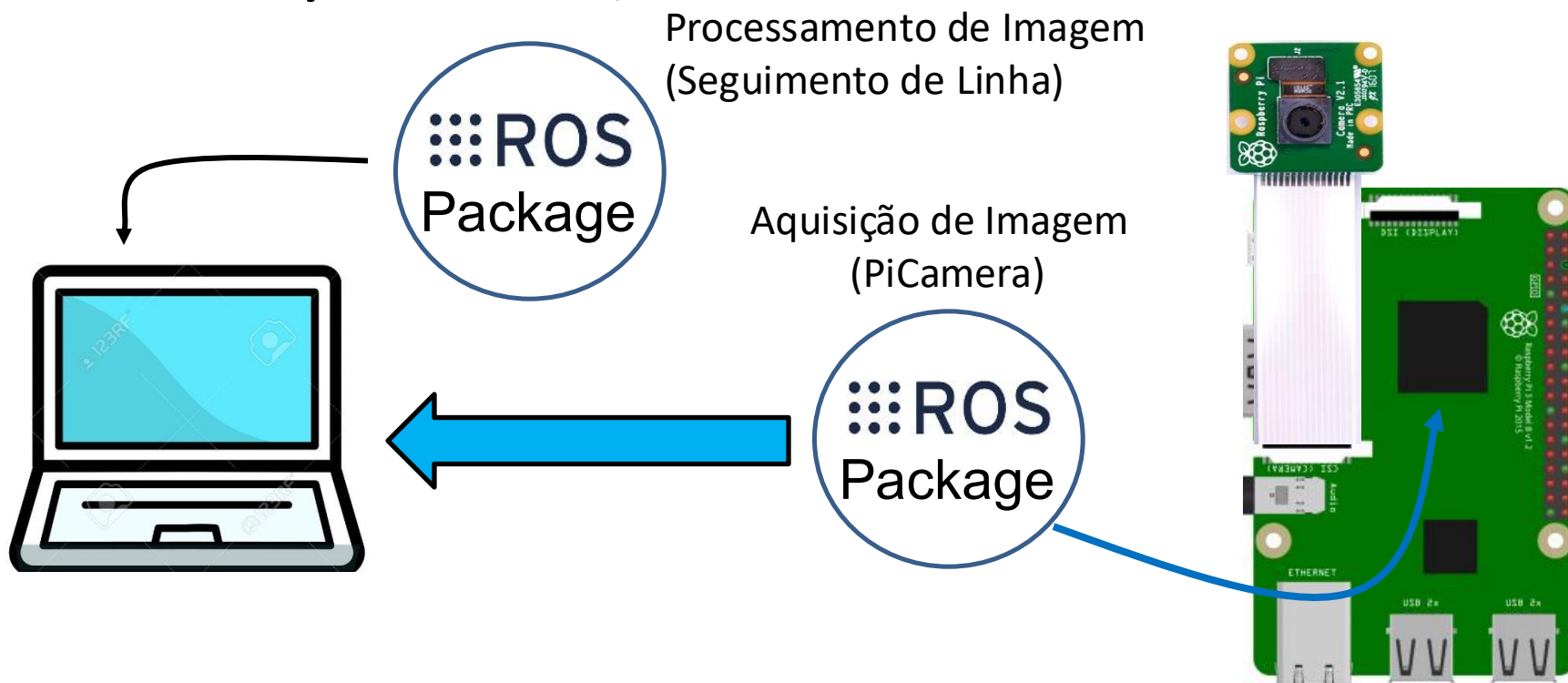
Cálculos de comandos de velocidade

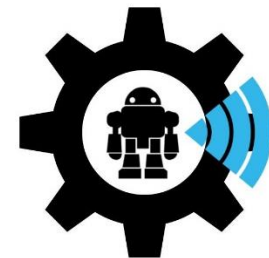




1. INTRODUÇÃO AO ROS

- Principais características:
 - Abstração de *hardware*;

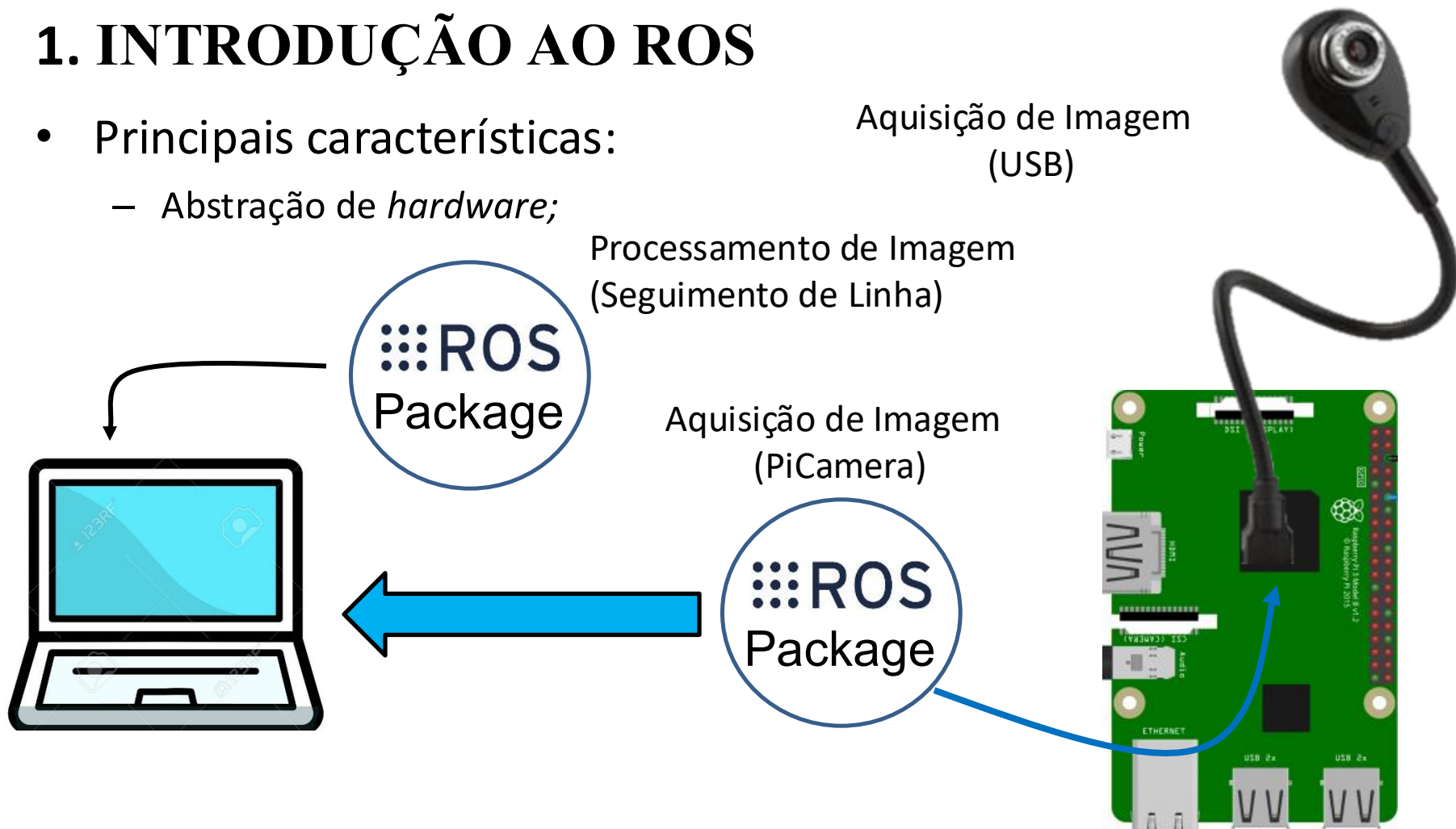


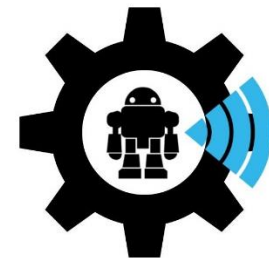


1. INTRODUÇÃO AO ROS

- Principais características:

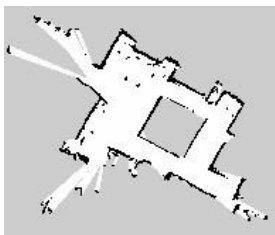
- Abstração de *hardware*;



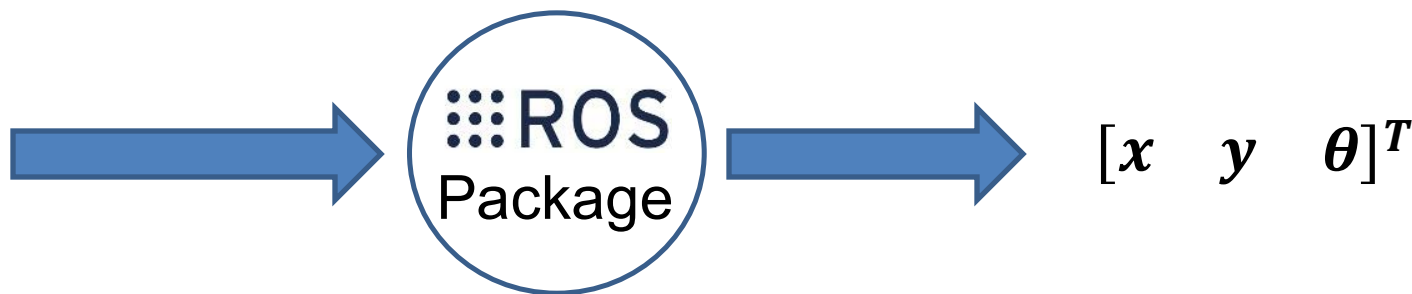


1. Introdução ao ROS

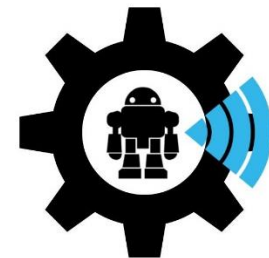
- Principais características:
 - Abstração de *hardware*;
 - Abstração de algoritmos complexos;



Algoritmo de Localização



$$[x \quad y \quad \theta]^T$$

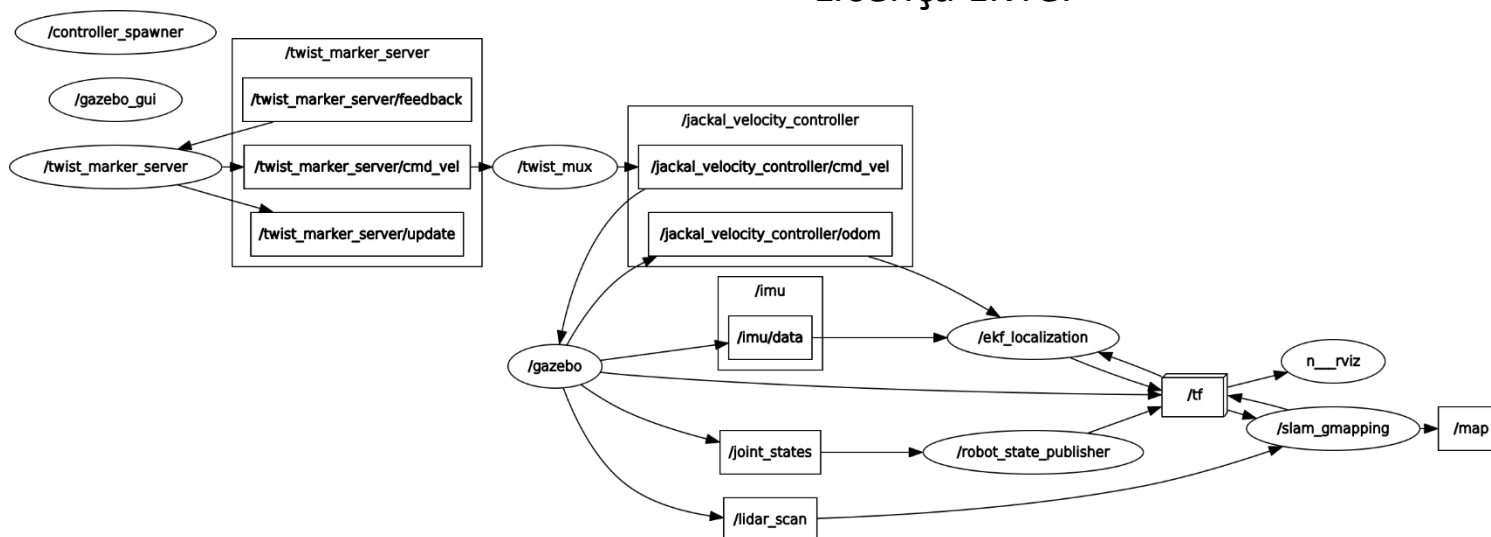


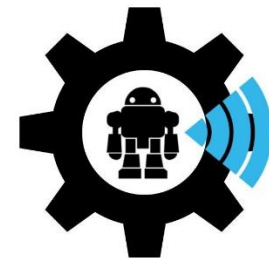
1. Introdução ao ROS

- Principais características:

- Abstração de *hardware*;
- Abstração de algoritmos complexos;
- Comunicação entre processos;

- Paralelismo de processos;
- Suporte a várias linguagens (Python, C++, JavaScript, Lisp, MATLAB, LUA e etc);
- Leve; e
- Licença Livre.





ROS

navigation

task executive

visualization

simulation

perception

control

planning

data logging

message passing

device drivers

real-time capabilities

web browser

email client

window manager

memory management

process management

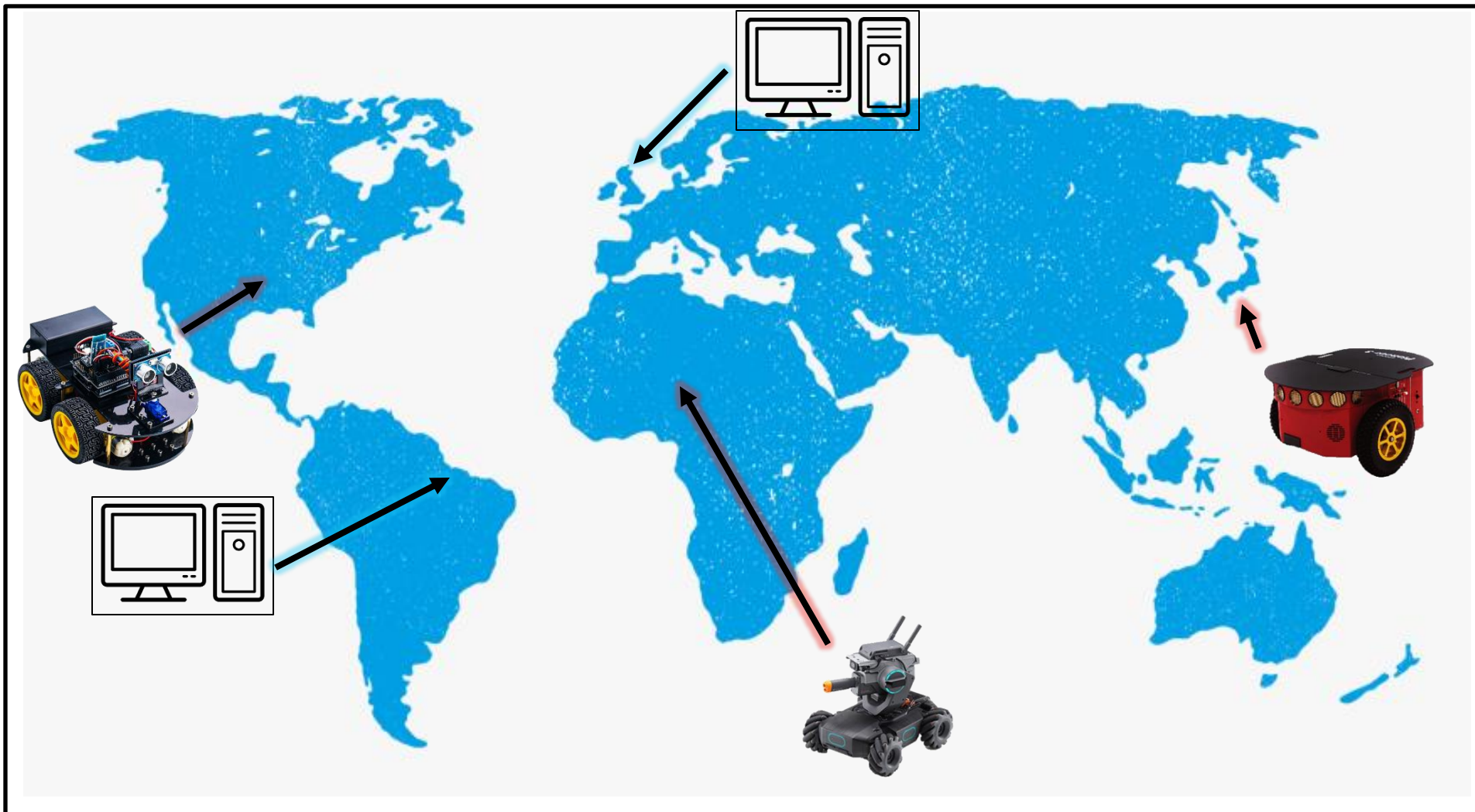
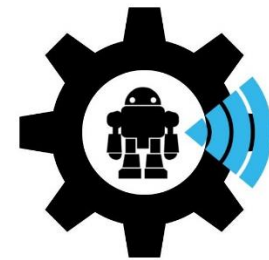
scheduler

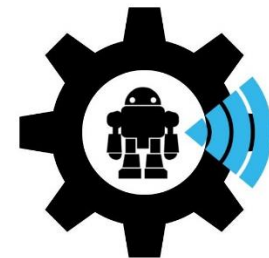
device drivers

file system

OS

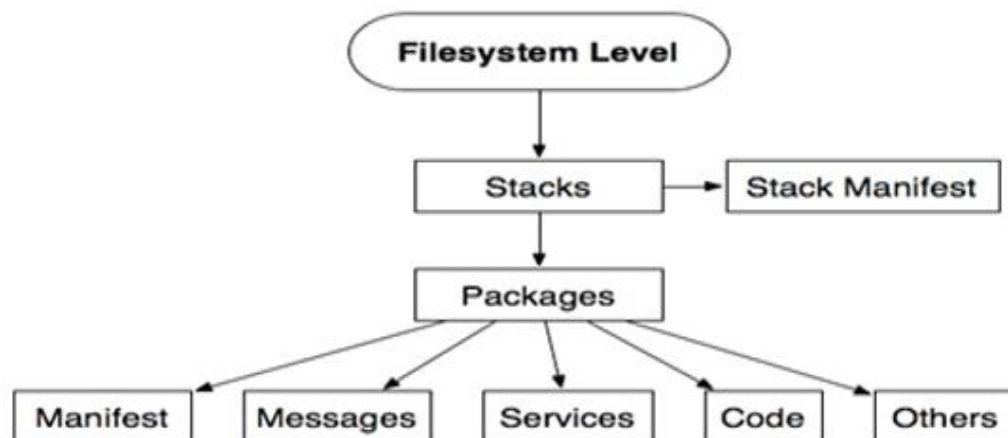






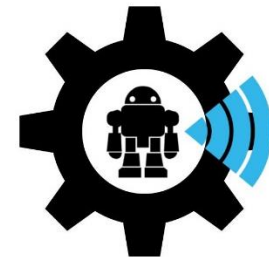
2. Nível Arquivo de Sistemas

- Este nível é responsável pela estrutura interna do ROS, de que forma as pastas estão organizadas no disco rígido e os arquivos necessários para a sua operação.



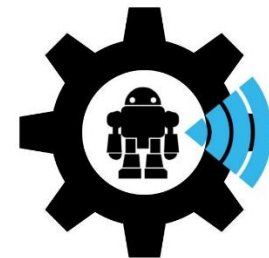
- Fonte: <http://wiki.ros.org/ROS/Concepts>

```
roseli
├── CMakeLists.txt
├── include
│   └── roseli
├── launch
│   └── roseli.launch
├── msg
│   └── PointVector.msg
├── package.xml
├── scripts
│   ├── creating_graph1.py
│   ├── followerbot.py
│   ├── map_saver.py
│   ├── primitivas.py
│   ├── readtag.py
│   ├── ros_SE.py
│   ├── test3.py
│   └── test_talker.py
├── src
│   ├── driveline1.cpp
│   ├── ImageConverter.cpp
│   ├── ImageConverter.h
│   ├── main.cpp
│   ├── move_md25_3.cpp
│   ├── movimentofunctions.c
│   ├── movimentofunctions.o
│   ├── movimento.h
│   └── teste1ros.cpp
└── srv
    ├── CreateMap.srv
    ├── GetOdom.srv
    ├── ResetEnc.srv
    └── sonar_server.srv
```



3. Nível de Comunidade

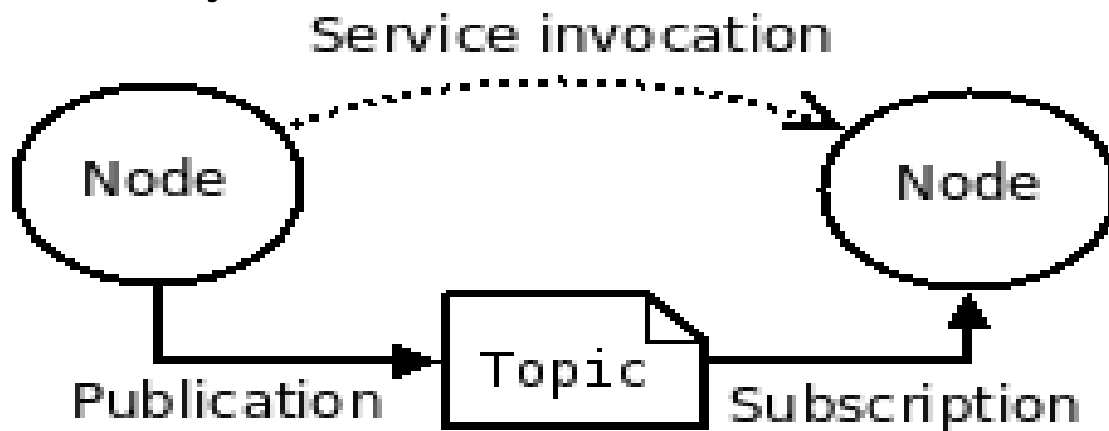
- É ao nível de comunidade que torna possível fazer a troca de *software* e conhecimento do ROS. Os principais canais em que os usuários interagem são:
 - ROS Answers (<https://answers.ros.org/questions/>)
 - ROS Wiki (<http://wiki.ros.org/>)
 - ROS Discourse (<https://discourse.ros.org/>)
- Fonte: <http://wiki.ros.org/ROS/Concepts>



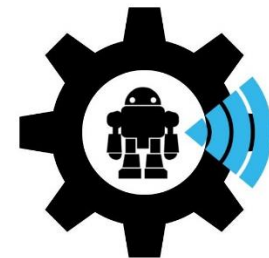
AULA 1 (RMP 01) – Introdução ao ROS

4. Nível Grafo Computacional

- Este nível está relacionado com a rede de comunicação entre processos executados em paralelo pelo ROS em um computador ou entre vários computadores heterogêneos. O grafo computacional é constituído por vértices ou nós, que representam os processos executáveis do sistema, e pelas arestas, que informam as comunicações estabelecidas entre esses processos.
- Existem basicamente duas formas de realizar comunicação entre processador na rede ROS: através de tópicos e de serviços.

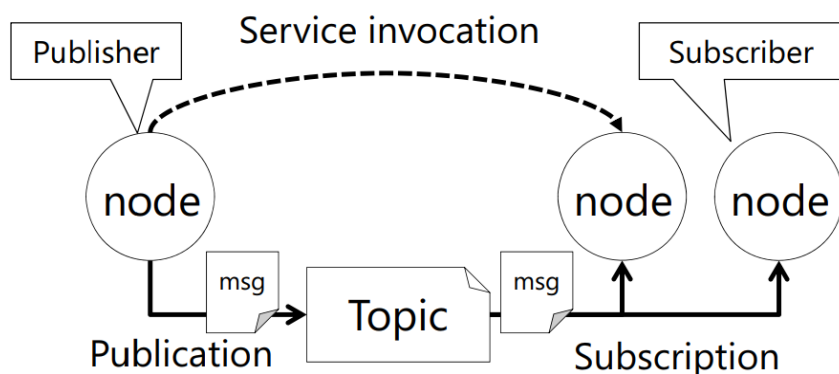


- Fonte: <http://wiki.ros.org/ROS/Concepts>



4. Nível Grafo Computacional

- A comunicação através de tópicos segue a estrutura de *publisher/subscriber* (publicador/consumidor). O nó *publisher* é aquele que transmite informações através do barramento enquanto o *subscriber* adquire as informações que foram transmitidas. Este tipo de troca de mensagens é unidirecional e de fluxo contínuo.
- Para que a comunicação seja de fato estabelecida os nós envolvidos devem compartilhar o mesmo tipo de mensagem (.msg) no mesmo tópico.



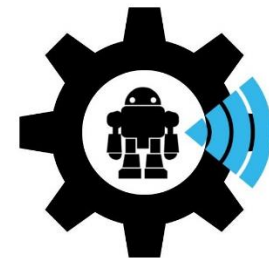
geometry_msgs/Point.msg

```
float64 x
float64 y
float64 z
```

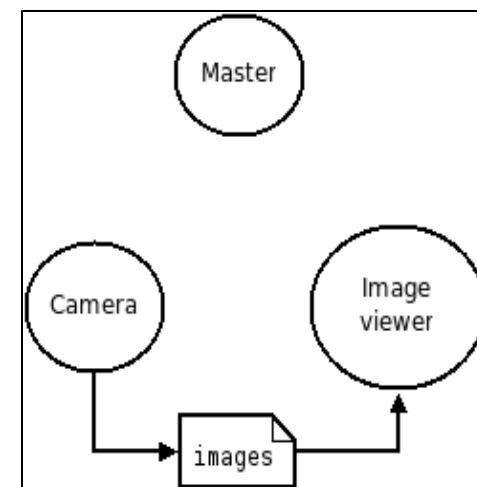
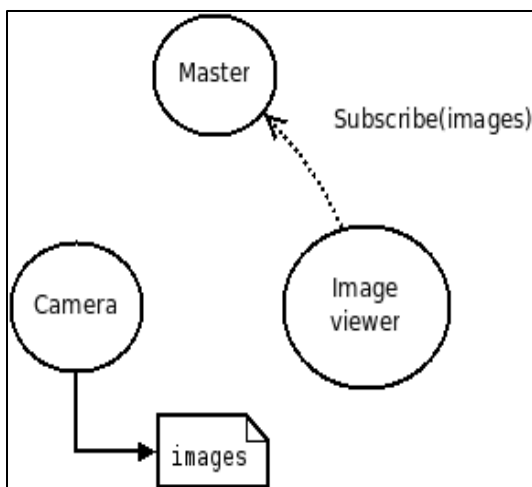
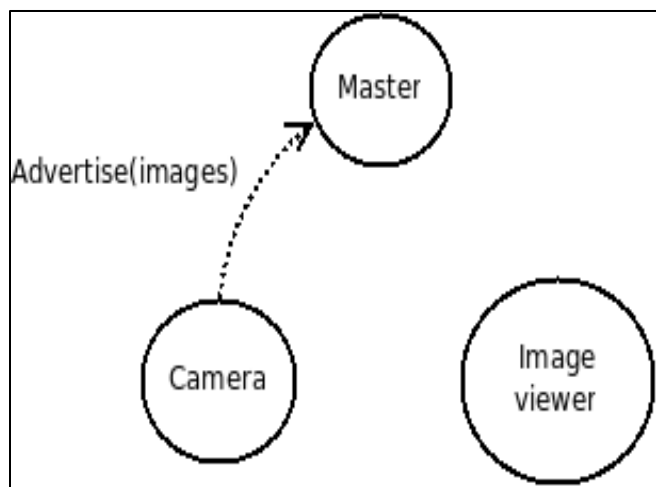
geometry_msgs/PoseStamped.msg

```
std_msgs/Header header
uint32 seq
time stamp
string frame_id
geometry_msgs/Pose pose
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Quaternion
    orientation
      float64 x
      float64 y
      float64 z
      float64 w
```

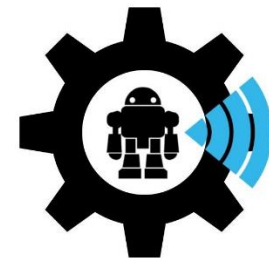
- Fonte: <http://wiki.ros.org/ROS/Concepts>



4. Nível Grafo Computacional

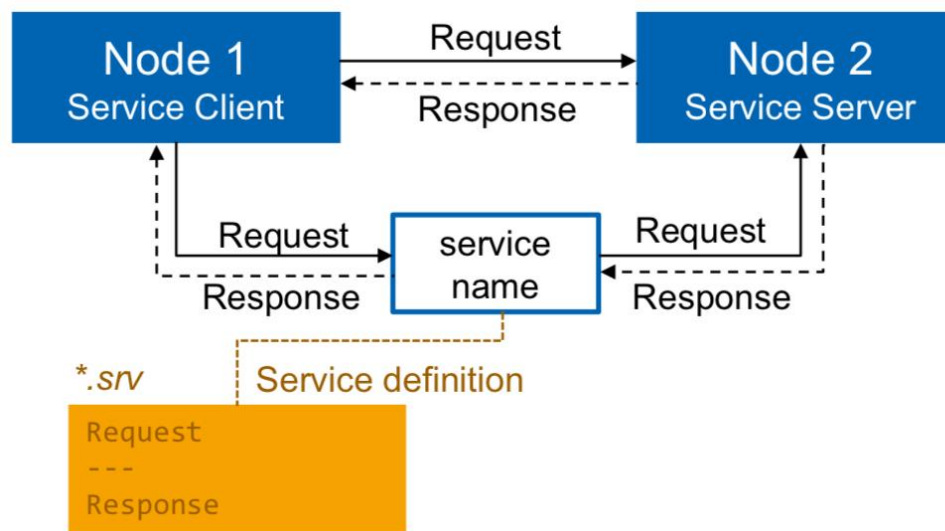


- Fonte: <http://wiki.ros.org/ROS/Concepts>

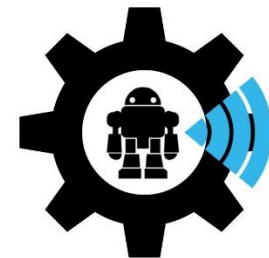


4. Nível Grafo Computacional

- A comunicação via serviço segue a estrutura cliente/servido. O nó cliente é aquele que envia uma requisição para o nó servidor, que por sua vez computa o pedido e retorna uma resposta ao cliente. Este tipo de comunicação tem a característica de ser bidirecional e com interação pedido resposta.



- Fonte: <http://wiki.ros.org/ROS/Concepts>



5. Instalação do ROS

- O ROS possui uma série de distribuições que são lançadas todo ano. Atualmente, a empresa que oferece o ROS sugere a instalação de duas distribuições por serem as últimas com suporte LTS (*Long-Term Suport*):

- ROS Noetic
- ROS Melodic Morenia

- O ROS oferece suporte para diversas plataformas, como: Ubuntu (recomendado), Debian, Windows, macOS, entre outros.

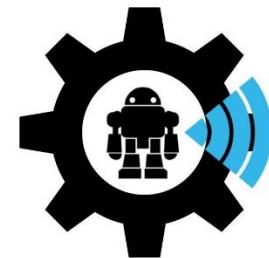
- Fonte: <http://wiki.ros.org/Distributions>

Distro	Release date	Poster	Tuturtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)



5. Instalação do ROS

- Os seguintes passos devem ser executados para instalar o ROS em uma máquina Ubuntu (seja como sistema host de computador ou na máquina Virtual):
 - `sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'`
 - `sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654`
 - `sudo apt update`
 - `sudo apt install ros-noetic-desktop-full`
 - `sudo rosdep init`
 - `rosdep update`
 - `echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc`
 - `source ~/.bashrc`
- Fonte: <http://wiki.ros.org/melodic/Installation/Ubuntu>



5. Instalação do ROS

- A instalação estará completa se a seguinte tela aparecer quando o comando <roscore> for executado:

```
roscore http://ubuntu:11311/
ubuntu@ubuntu:~$ roscore
... logging to /home/ubuntu/.ros/log/9b337e2e-23d0-11e9-a3a7-000c293e7ce6/ros-lau
nch-ubuntu-14912.log
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:34032/
ros_comm version 1.12.14

SUMMARY
=====

PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.14

NODES

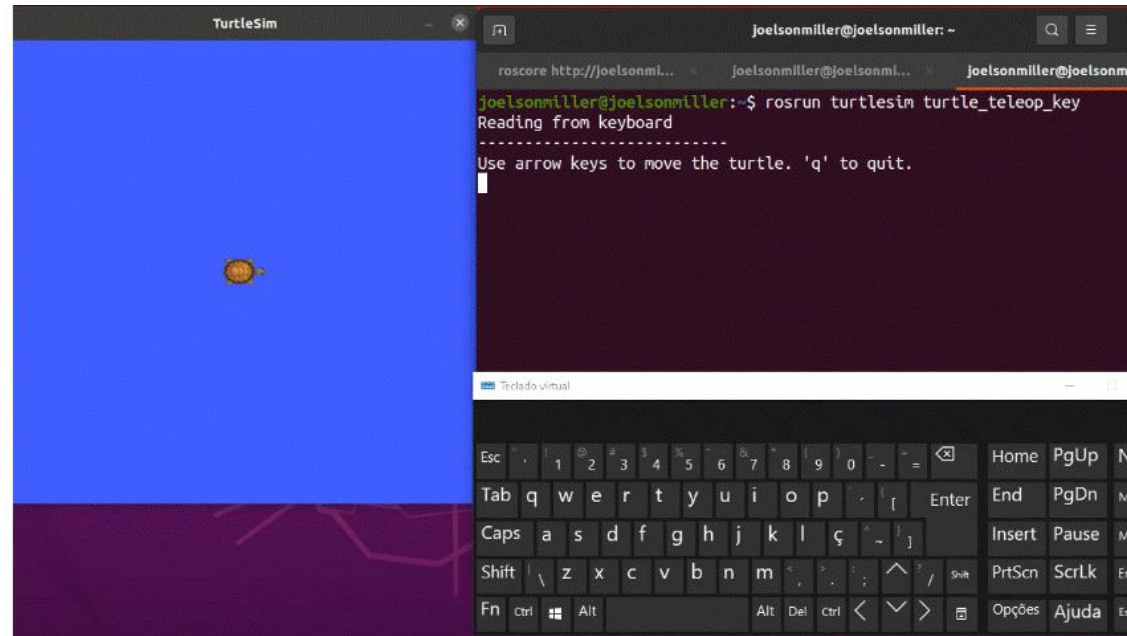
auto-starting new master
process[master]: started with pid [14922]
ROS_MASTER_URI=http://ubuntu:11311/

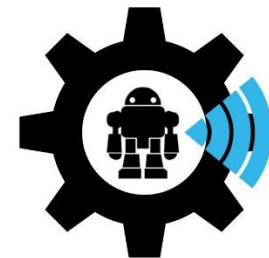
setting /run_id to 9b337e2e-23d0-11e9-a3a7-000c293e7ce6
process[rosout-1]: started with pid [14935]
started core service [/rosout]
█
```



5. Instalação do ROS

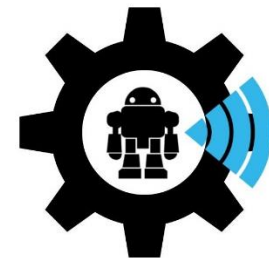
- Os comandos a seguir são relacionados a mostrar trocas básicas de informações entre processos. Neste caso se trata de um simulador simples de movimento 2D e um processor que envia comandos de movimentos a partir das setas do teclado do computador.
 - `roslaunch turtlesim turtlesim_node`
 - `roslaunch turtlesim turtle_teleop_key`





5. Instalação do ROS

- **roscore**: inicia o ROS mestre.
- **roslaunch** <nome do pacote> <nome do executável> <param>:=<valor do param> : possibilita a execução de um nó de um determinado pacote.
- **roslaunch** <nome do pacote> <nome do arquivo .launch>: possibilita a execução de vários nós em um único comando.
- **roscall** <subcomando> <nome do nó>:
 - list: lista os nós ativos na tela
 - info: mostra na tela informações sobre um nó
 - kill: para a execução de um nó
- **rostopic** <subcomando> <nome do tópico>:
 - list: lista os tópicos presentes na rede
 - echo <nome do tópico>: mostra na tela os dados de um tópico.
 - pub <nome do tópico> <tipo da msg> [dados...]: publica em um tópico uma informação
 - info <nome do tópico>: mostra na tela informações sobre um tópico (tipo da msg, nós conectados a ele e etc..)
 - find <tipo da msg>: lista na tela os tópicos ativos que possuem determinado tipo de mensagem
 - hz <nome do tópico>: mostra na tela a frequência de publicação de um tópico.



5. Instalação do ROS

- **rosparam <subcomando> <nome do parâmetro>:**
 - list <nome do parâmetro>: lista os parâmetro ativos no ROS
 - get <nome do parâmetro>: mostra na tela o valor atual de um parâmetro
 - set <nome do parâmetro> [dados...]: configura o valor de um parâmetro
 - delete <nome do parâmetro>: apaga o valor de um parâmetro
 - load <nome do arquivo .yaml> <nome do parâmetro> : configura vários parâmetros de uma só vez.
- **rosmmsg <subcomando> <tipo da mensagem>:**
 - show <tipo da mensagem>: mostra a estrutura de um tipo de mensagem
 - list: lista na tela todos as mensagens que existem no ROS
 - package <nome do pacote>: lista todas as mensagens de um pacote